Министерство образования Республики Беларусь Учреждение образования
«Брестский государственный технический университет» Кафедра интеллектуальных информационных технологий

Лабораторная работа №2
По дисциплине «Интеллектуальный анализ данных»
Тема: «Автоэнкодеры»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-25
Милевский А.Д.
Проверил:
Андренко К.В.

Брест 2026

Цель работы: научиться применять автоэнкодеры для осуществления визуализации данных и их анализа.

Общее задание

Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент с использованием нейросетевой модели автоэнкодера (с двумя и тремя нейронами в среднем слое);

Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;

Реализовать метод t-SNE для визуализации данных (использовать также 2 и 3 компонента), построить соответствующую визуализацию;

Применить к данным метод PCA (2 и 3 компонента), реализованный в ЛР №1, сделать выводы;

Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Класс
9	**Wine Quality (white)**	**quality**

Ход работы

Код программы:

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'  # Отключаем лишние предупреждения TensorFlow

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model

print("--- Загрузка и подготовка данных ---")
link = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.csv"
wine = pd.read_csv(link, sep=';')
X = wine.drop('quality', axis=1).values
y = wine['quality'].values

scaler = StandardScaler()
X = scaler.fit_transform(X)

print(f"Размерность данных: {X.shape}")
print(f"Уникальные классы (quality): {np.unique(y)}\n")

def draw_graph(points, classes, header, label_x, label_y):
    plt.figure(figsize=(9, 6))
    sp = plt.scatter(points[:, 0], points[:, 1], c=classes,
                     cmap='viridis', s=12, alpha=0.75)
    plt.colorbar(sp, label='quality')
    plt.title(header)
    plt.xlabel(label_x)
    plt.ylabel(label_y)
    plt.grid(alpha=0.3)
    plt.show()

# --- Автоэнкодер ---
def make_autoencoder(n_features, n_hidden):
    inp = Input(shape=(n_features,))
    bottleneck = Dense(n_hidden, activation='relu')(inp)
    out = Dense(n_features, activation='linear')(bottleneck)

    full = Model(inp, out)
    enc = Model(inp, bottleneck)

    full.compile(optimizer='adam', loss='mse')
    # Сохраняем историю обучения
    history = full.fit(X, X, epochs=50, batch_size=32, shuffle=True, verbose=0)
    return enc, full, history

print("--- Обучение автоэнкодера (2 нейрона) ---")
ae2, full_ae2, hist2 = make_autoencoder(X.shape[1], 2)
latent2 = ae2.predict(X, verbose=0)
```

```
final_loss_2 = full_ae2.evaluate(X, X, verbose=0)
print(f"Итоговая ошибка (MSE): {final_loss_2:.4f}")

print("\n--- Обучение автоэнкодера (3 нейрона) ---")
ae3, full_ae3, hist3 = make_autoencoder(X.shape[1], 3)
latent3 = ae3.predict(X, verbose=0)
final_loss_3 = full_ae3.evaluate(X, X, verbose=0)
print(f"Итоговая ошибка (MSE): {final_loss_3:.4f}\n")

# --- График сходимости ---
print("--- Построение графика сходимости ---")
plt.figure(figsize=(9, 6))
plt.plot(hist2.history['loss'], label='Autoencoder (2 нейрона)', linewidth=2)
plt.plot(hist3.history['loss'], label='Autoencoder (3 нейрона)', linewidth=2)
plt.title('Сходимость обучения автоэнкодеров')
plt.xlabel('Эпоха')
plt.ylabel('MSE loss')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# --- Визуализация автоэнкодеров ---
draw_graph(latent2, y, 'Autoencoder, сжатие до 2 признаков',
        'признак 1', 'признак 2')

draw_graph(latent3[:, :2], y, 'Autoencoder, сжатие до 3 признаков (первые 2 оси)',
        'признак 1', 'признак 2')

# --- t-SNE ---
print("\n--- Применение t-SNE ---")
tsne2 = TSNE(n_components=2, random_state=42, perplexity=30)
res_tsne2 = tsne2.fit_transform(X)
print("t-SNE (2 компоненты) готов")
draw_graph(res_tsne2, y, 't-SNE, 2 компоненты', 't-SNE 1', 't-SNE 2')

tsne3 = TSNE(n_components=3, random_state=42, perplexity=30)
res_tsne3 = tsne3.fit_transform(X)
print("t-SNE (3 компоненты) готов")
draw_graph(res_tsne3[:, :2], y, 't-SNE, 3 компоненты (первые 2 оси)',
        't-SNE 1', 't-SNE 2')

# --- PCA ---
print("\n--- Применение PCA ---")
pca2 = PCA(n_components=2)
res_pca2 = pca2.fit_transform(X)
print(f"Доля объясненной дисперсии (2 компоненты): {pca2.explained_variance_ratio_.sum():.4f}")
draw_graph(res_pca2, y, 'PCA, 2 компоненты', 'PC1', 'PC2')

pca3 = PCA(n_components=3)
res_pca3 = pca3.fit_transform(X)
print(f"Доля объясненной дисперсии (3 компоненты): {pca3.explained_variance_ratio_.sum():.4f}")
draw_graph(res_pca3[:, :2], y, 'PCA, 3 компоненты (первые 2 оси)',
        'PC1', 'PC2')

print("\nВсе графики успешно построены. Работа завершена.")
```

Вывод программы:

```
C:\Users\artem\PyCharmMiscProject\.venv\Scripts\py
WARNING: All log messages before absl::InitializeL
I0000 00:00:1794507779.215231   23068 port.cc:153]
WARNING: All log messages before absl::InitializeL
I0000 00:00:1794507780.955522   23068 port.cc:153]
--- Загрузка и подготовка данных ---
Размерность данных: (4898, 11)
Уникальные классы (quality): [3 4 5 6 7 8 9]

--- Обучение автоэнкодера (2 нейрона) ---
Итоговая ошибка (MSE): 0.5665

--- Обучение автоэнкодера (3 нейрона) ---
Итоговая ошибка (MSE): 0.4579

--- Построение графика сходимости ---

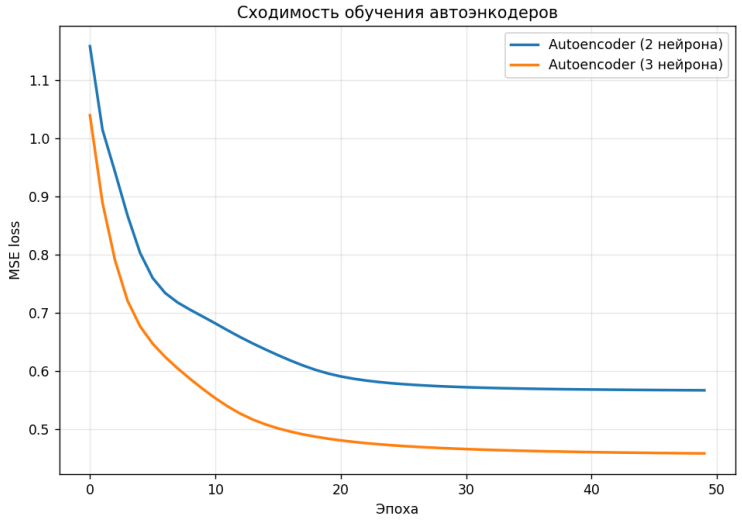
--- Применение t-SNE ---
t-SNE (2 компоненты) готов
t-SNE (3 компоненты) готов

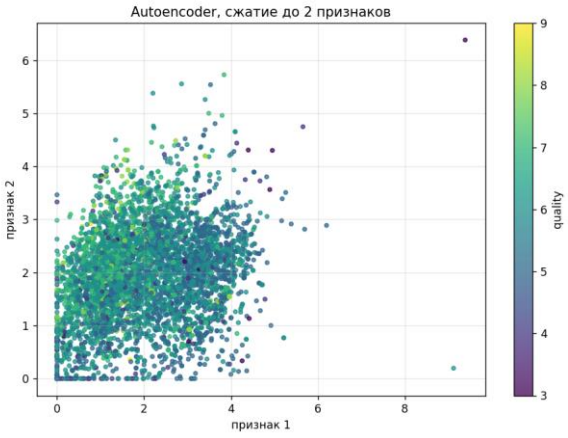
--- Применение PCA ---
Доля объясненной дисперсии (2 компоненты): 0.4361
Доля объясненной дисперсии (3 компоненты): 0.5472

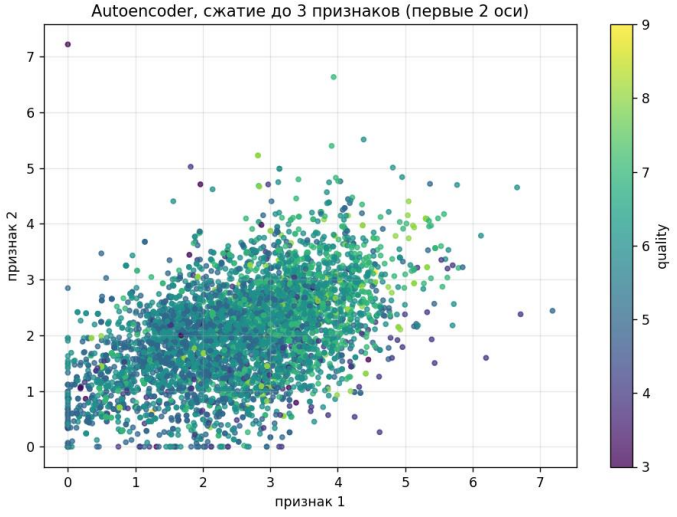
Все графики успешно построены. Работа завершена.

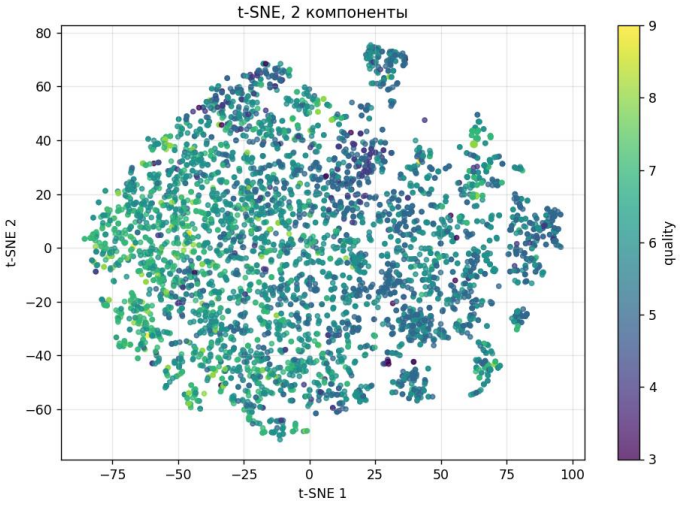
Process finished with exit code 0
```

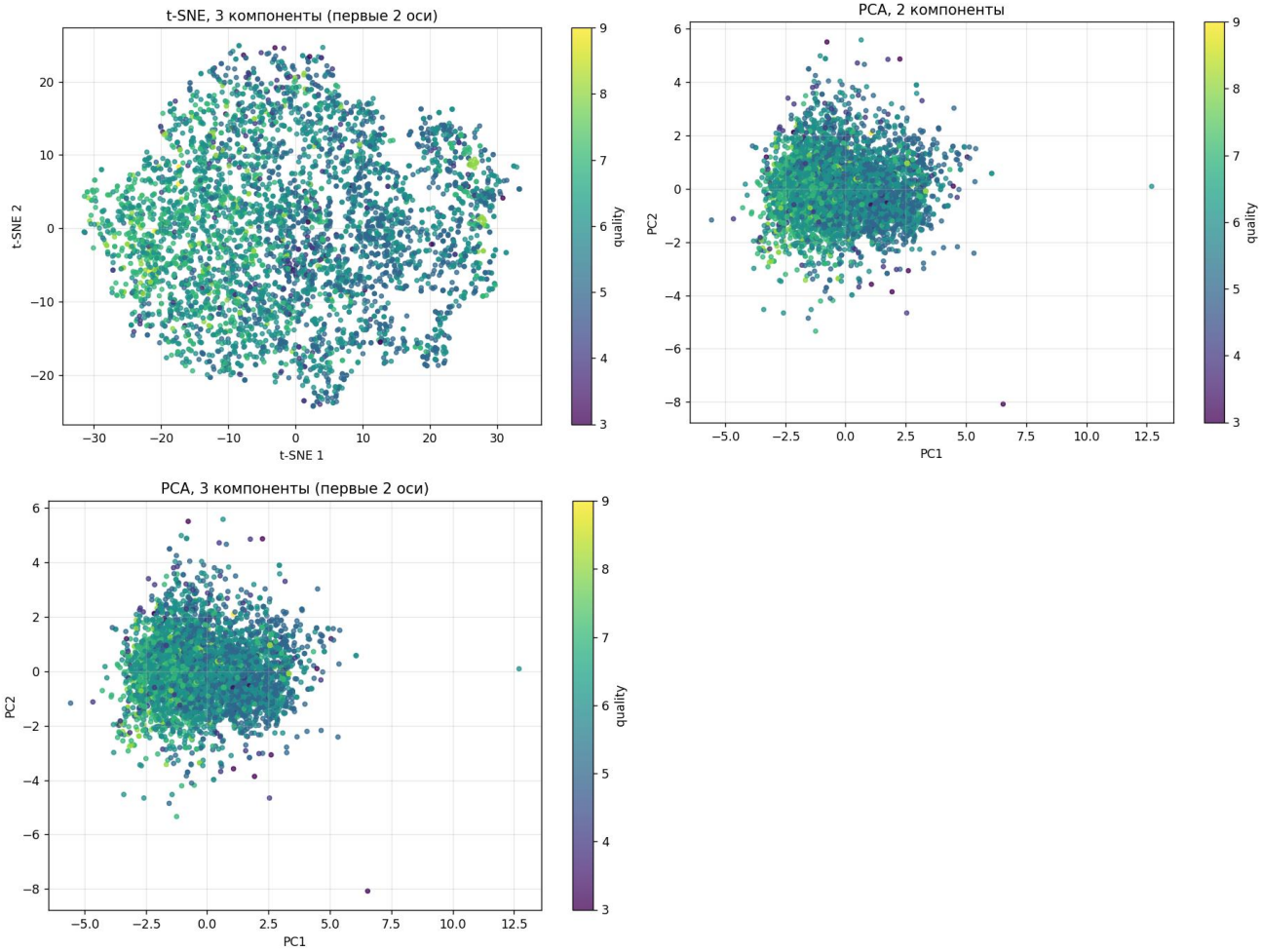
Графики:











В ходе лабораторной работы к датасету **Wine Quality (white)** (4898 объектов, 11 физико-химических признаков) были применены три метода снижения размерности: автоэнкодер, PCA и t-SNE.

Автоэнкодер с архитектурой **11→2/3→11** (и функцией потерь MSE) показал снижение ошибки с **0.5665** (для 2 компонент) до **0.4579** (для 3 компонент), что говорит о более точном восстановлении данных при увеличении скрытого слоя. Однако визуальный анализ латентного пространства показал, что классы качества вина (от 3 до 9) сильно перемешаны, и модель не смогла линейно разделить их в узком «бутылочном горлышке».

Метод **PCA** сохранил **43,61%** дисперсии для двух компонент и **54,72%** для трёх. Графики показали единое плотное облако точек без четких границ между классами, что отражает лишь глобальную линейную структуру данных.

Метод **t-SNE** (при perplexity=30) обеспечил наилучшее визуальное разделение данных на локальном уровне. В отличие от PCA и автоэнкодера, он разбил облако на множество отдельных «островков», выявив скрытые нелинейные связи между химическими показателями вин. Тем не менее, строгих границ между конкретными оценками качества (например, 5 и 6) нет, так как органолептические свойства вина представляют собой непрерывный спектр.

Сравнение методов: автоэнкодер гибко формирует нелинейное латентное пространство, PCA сохраняет общую дисперсию (но только линейно), а t-SNE лучше всего подходит для поиска локальных кластеров и кластерного анализа.

Вывод: научился применять автоэнкодеры, PCA и t-SNE для визуализации и анализа многомерных данных на примере задачи оценки качества вина.